

BÁO CÁO KỸ THUẬT

Phần B — Graph RAG & LLM Serving (40%)

B1: Graph RAG — Knowledge Base (25%) — B2: LLM Serving & Optimization (25%)

Đề bài: MAS-LLM-2026-FINAL
Phần cứng thực đo: NVIDIA RTX 4050 Laptop 6GB VRAM (khác giả định đề bài: RTX 3060 12GB)

Mục lục

- 1 1. Kiến trúc hệ thống (B1 Graph RAG + B2 LLM Serving)
- 2 2. B1 — Graph RAG: Database Schema, Vector Index & Hybrid Search, Data Ingestion
- 3 3. B2 — LLM Serving: Serving Infrastructure, Streaming SSE, Caching, Benchmark TTFT
- 4 4. Phân tích thất bại (Failure Analysis)
- 5 5. Kết luận và khuyến nghị

1. Kiến trúc hệ thống

Phần B gồm 2 module theo đúng cấu trúc đề bài: **B1 - Graph RAG - Knowledge Base** (Neo4j, vector index, hybrid search, ingestion pipeline, 25%) và **B2 - LLM Serving & Optimization** (SGLang serving, streaming SSE, caching đa tầng, benchmark TTFT, 25%). Router/Multi-Agent (Phần A) sử dụng lại B1 (qua `CachedKnowledgeBase.hybrid_search()`) làm nguồn tri thức cho 3 agent chuyên biệt.

```
User Query
|
v
[B1.2] embed_one(query) -- Qwen3-Embedding-0.6B
|
v
[B1.2] Dual-Domain Search (Neo4j Vector Index: menu_embedding + chunk_embedding)
|
v
[B1.2] Graph Expansion (NEXT/PREV liên ke + MENTIONS entity chung)
|
v
[B1.2] Late Reranking (BAAI/bge-reranker-v2-m3, threshold >= 0.7)
|
v
[B2.3] GraphCache (TTL+LRU, tránh gọi lại Neo4j cho câu hỏi trùng)
|
v
[B2.1] SGLang Generator (AWQ INT4) -- augmented prompt (context từ B1)
|
v
[B2.2] SSE Streaming (token + clause-level) --> Response
```

2. B1 — Graph RAG: Knowledge Base

2.1. B1.1 — Graph Database Schema

Yêu cầu B1.1	Trạng thái	Bằng chứng
Graph Database (Neo4j)	PASS	Neo4j chạy qua Docker
Node types: MenuItem, Chunk, Entity	PASS	hybrid_search.py truy v
Relationships: NEXT, MENTIONS, BELONGS_TO	PASS 1 phần	NEXT và MENTIONS xá
>= 100 MenuItem, >= 150 FAQ chunk, >= 50 Document chunk	CHƯA ĐỦ BẰNG CHỨNG	Đếm được tổng 220 no

Ghi chú: con số 220 chunk xác nhận database CÓ dữ liệu thật (không rỗng), nhưng chưa breakdown theo từng loại node để đối chiếu chính xác với 3 ngưỡng (>=100/>=150/>=50) — cần chạy thêm MATCH (m:MenuItem) RETURN count(m) và phân loại Chunk theo source/category trước khi kết luận PASS/FAIL chính thức.

2.2. B1.2 — Vector Index & Hybrid Search

Pipeline đúng 3 bước theo đề bài: **Dual-Domain Search** (vector index menu_embedding + chunk_embedding, Neo4j native db.index.vector.queryNodes) → **Graph Expansion** (duyệt NEXT/PREV chunk liền kề + MENTIONS entity chung) → **Late Reranking** (cross-encoder BAAI/bge-reranker-v2-m3, lọc theo ngưỡng relevance >= 0.7, đúng ngưỡng đề bài).

Yêu cầu B1.2	Trạng thái	Bằ
Vector Index: menu_embedding + chunk_embedding	PASS	Cy
Embedding model đa ngôn ngữ (Qwen3-Embedding-0.6B)	PASS	em
Reranker (BGE Reranker v2 hoặc tương đương)	PASS	BA
Hybrid Search: Dual-Domain -> Graph Expansion -> Late Reranking (threshold >= 0.7)	PASS — có demo thật	CL
Top-5 precision >= 80% trên 20 câu hỏi test	CHƯA ĐO	Ch
Graph Expansion tăng recall >= 15% so với vector search thuần	CHƯA ĐO	Ch

Phát hiện quan trọng khi test thật đêm nay: ngưỡng rerank >= 0.7 hoạt động đúng như thiết kế — với câu hỏi diễn giải mơ hồ ("Có món nào mát mát cho mùa hè không..."), toàn bộ candidate đều bị lọc dưới 0.7 (không có gì lọt qua), trong khi câu hỏi khớp rõ với dữ liệu thật ("cà phê sữa đá") cho rerank score rất cao (0.99). Điều này cho thấy pipeline B1.2 tự nó hoạt động đúng — nhưng hệ quả ở tầng Generator (B2) khi nhận 0 context lại là vấn đề nghiêm trọng, xem phần Zero Hallucination bên dưới.

2.3. B1.3 — Data Ingestion Pipeline

Yêu cầu B1.3	Trạng thái	Bằng chứng
Ingest Menu CSV / FAQ CSV / PDF-DOCX (semantic chunking + entity extraction)	CHƯA KIỂM CHỨNG	Không có b
Entity Deduplication (Jaccard similarity >= 0.85)	CHƯA KIỂM CHỨNG	Chưa có bằ
Watch Mode (tự detect file mới, ingest realtime)	CHƯA KIỂM CHỨNG	Chưa có bằ

Ghi chú trung thực: B1.3 là phần duy nhất của B1 chưa được kiểm tra trong phiên làm việc đêm nay (trọng tâm đêm nay là B1.2 + B2.2 + B2.4 phục vụ demo video). Dữ liệu đã có sẵn

trong Neo4j (220 chunk) chứng minh ingestion ĐÃ chạy được ít nhất 1 lần thành công trước đó, nhưng cần xem lại code/log ingestion cụ thể trước khi báo cáo PASS chính thức cho B1.3.

2.4. Zero Hallucination (tiêu chí nghiệm thu B1, 5%) — FAIL nghiêm trọng, có bằng chứng thật

Bước	Kết quả thật đo được
Câu hỏi	"Có món nào mát mát cho mùa hè không, giá bao nhiêu vậy em?"
B1.2 Hybrid Search trả về	0 chunk (đúng thiết kế — không có candidate nào đạt ngưỡng rerank >= 0.7 vì câu
Context đưa vào Generator	RỖNG (chuỗi context block trống)
Câu trả lời Generator sinh ra	"Có món mát mát cho mùa hè là cà phê đá và kem tươi. Giá của mỗi cốc là 55,000đ."

Đây là bằng chứng cụ thể, tái hiện được, cho thấy tiêu chí 'Zero hallucination' của B1 (và cả B2.4 'LLM trả lời hoàn toàn dựa trên context từ RAG, không bịa thông tin đặc biệt về giá, menu, chính sách') đang FAIL. Nguyên nhân gốc: hệ thống hiện tại KHÔNG có bước kiểm tra 'nếu context rỗng thì từ chối trả lời/nói không biết' — system prompt hiện tại chỉ dặn 'Nếu thông tin không có, đừng bịa' bằng lời nhắc (prompt-based guardrail), nhưng lời nhắc suông không đủ để ngăn hallucination khi context thực sự rỗng — mô hình generator (AWQ INT4, quy mô nhỏ) vẫn ưu tiên sinh câu trả lời 'nghe hợp lý' hơn là thừa nhận không biết.

3. B2 — LLM Serving & Optimization

3.1. B2.1 — Serving Infrastructure

Yêu cầu B2.1	Trạng thái	Bằng chứng
Serving qua SGLang hoặc vLLM	PASS (SGLang)	python3 -m sg
Dual-model trên 1 GPU (Router 0.5-1.5B AWQ + Generator 7B AWQ)	FAIL — kiến trúc khác đề bài	Generator serv
mem-fraction-static / context-length / schedule-policy=lpm tuning	PASS 1 phần	mem-fraction-s
Container hóa bằng Docker Compose	FAIL — CHƯA CÓ	Toàn bộ hệ thố

3.2. B2.2 — Streaming & Token-level Output

Yêu cầu B2.2	Trạng thái	Bằng chứng
True token streaming qua SSE	PASS	api_server.py: StreamingResp
Xử lý đúng sentinel [DONE] (literal string, không bọc JSON)	PASS	sse_generator() kết thúc bằng
Clause-level streaming cho TTS	PASS	ClauseBuffer cắt theo CLAUSE
Đã test end-to-end qua chính endpoint /v1/chat/stream	CHƯA TEST TRỰC TIẾP	Đêm nay chỉ test lại logic stre

3.3. B2.3 — Caching Architecture

Layer cache (theo đề bài)	Trạng thái	Bằng chứng
Model Cache (Disk)	PASS (thiết kế)	ModelCache trong cache_layer
Graph Cache (Persistent DB / kết quả RAG)	PASS — có dùng thật trong demo	GraphCache (TTL 300s, LRU m
KV Cache (VRAM, prefix-sharing)	PASS (đúng theo khuyến nghị đề bài)	Không tự viết lại — dùng cơ ch
Session Cache (In-Memory)	PASS	Tái sử dụng SessionStore đã lầ
Semantic Cache (bonus, similarity >= 0.95)	PASS (thiết kế, có self-test)	SemanticCache dùng real_emb

Khuyến nghị bổ sung bằng chứng: nên chạy python3 cache_layer.py trước khi nộp bài để có log PASS/FAIL cụ thể (assert đã viết sẵn trong code) cho cả 3 tầng Model/Graph/Semantic Cache, làm bằng chứng trực tiếp thay vì chỉ review code.

3.4. B2.4 — Benchmark & Latency Budget (số liệu thật đêm nay)

Đo trên demo GraphRAG + Streaming thật (câu hỏi "Cho em hỏi cà phê sữa đá size L giá bao nhiêu vậy?", model generator_awq_v2 AWQ INT4 qua SGLang, GPU RTX 4050 6GB thay vì RTX 3060 12GB đề bài giả định).

Chỉ số	Giá trị đo thật	Người
TTFT (Generator, có context GraphRAG augmented)	3209.7 ms	<=2
Tổng thời gian sinh câu trả lời (generation only)	4376.9 ms	-
Tổng Pipeline Latency (Retrieval + Generation, đã loại thời gian load model lần đầu)	Chưa tách riêng được	<=3

Throughput (tokens/giây, nhiều request đồng thời)	CHƯA ĐO	-
---	---------	---

Phân tích nguyên nhân TTFT cao: 3209.7ms cao hơn rất nhiều so với ngưỡng ngay cả khi tính đến chênh lệch phần cứng (RTX 4050 6GB vs RTX 3060 12GB). Các giả thuyết cần kiểm chứng thêm: (1) đây là request đầu tiên gửi tới model sau khi model vừa load — có thể dính chi phí warm-up/compile kernel lần đầu (hiện tượng đã quan sát thấy tương tự ở demo Phần A, nơi request đầu tiên mất tới 166 giây do retry qua 3 lần timeout); (2) system prompt đã được augment thêm context GraphRAG (dài hơn prompt gốc), làm tăng thời gian prefill trước khi sinh token đầu tiên; (3) AWQ INT4 dequant có thể có overhead ở bước prefill trên GPU cấp thấp. Cần đo lại TTFT ở request thứ 2, 3 trở đi (model đã warm hoàn toàn, không còn compile lần đầu) để có con số phản ánh đúng hiệu năng ổn định của hệ thống, thay vì con số bị nhiễu bởi cold-start.

4. Phân tích thất bại (Failure Analysis)

- **Zero Hallucination FAIL — có bằng chứng cụ thể, tái hiện được (finding nghiêm trọng nhất của Phần B):** khi Hybrid Search (B1.2) trả về 0 chunk (đúng thiết kế, do câu hỏi quá diễn giải không đạt ngưỡng rerank 0.7), Generator vẫn tự tin bịa ra câu trả lời "Có món mát mát cho mùa hè là cà phê đá và kem tươi. Giá của mỗi cốc là 55,000đ." — hoàn toàn không có căn cứ, sai cả tên món lẫn giá. Đây là vi phạm trực tiếp tiêu chí nghiêm thu quan trọng nhất theo đề bài ("Độ chính xác & Zero Hallucination: Model trả lời đúng, không bịa — đây là tiêu chí tối thượng", trang 15). Nguyên nhân: guardrail hiện tại chỉ là 1 câu nhắc trong system prompt ("Nếu thông tin không có, đừng bịa"), không có cơ chế kiểm tra bắt buộc (hard guardrail) ở tầng code khi context rỗng.
- **TTFT vượt xa ngưỡng đề bài (3209.7ms đo thật vs <=200ms Đạt):** ngay cả sau khi trừ hao chênh lệch phần cứng (RTX 4050 6GB vs RTX 3060 12GB), khoảng cách vẫn quá lớn (~16 lần). Nghi ngờ chính: đo ở request đầu tiên (cold-start/compile kernel lần đầu), cần đo lại ở các lần gọi sau để có số liệu ổn định — hiện tại CHƯA CÓ số liệu TTFT 'sạch' (warm) để đối chiếu công bằng với ngưỡng đề bài.
- **Kiến trúc Serving không đúng thiết kế 'dual-model qua SGLang':** đề bài yêu cầu cả Router (0.5-1.5B, AWQ) và Generator (7B, AWQ) đều serve qua SGLang trên cùng 1 GPU với VRAM budget rõ ràng cho từng model (Router ~1.1GB, Generator ~4.8GB). Thực tế hiện tại chỉ Generator serve qua SGLang; Router load trực tiếp qua transformers trong process Python, tranh chấp VRAM không kiểm soát với Generator/Embedding/Reranker — đây chính là nguyên nhân gốc của sự cố OOM nghiêm trọng đã ghi nhận ở soak test Phần A (mục 3.5, báo cáo A).
- **Docker Compose containerization — CHƯA CÓ:** đề bài yêu cầu rõ 'Toàn bộ hệ thống phải container hóa bằng Docker Compose, khởi động theo đúng thứ tự dependency'. Hiện tại chỉ Neo4j chạy qua Docker (container đơn lẻ, khởi động thủ công); SGLang Generator, Router, FastAPI (api_server.py) đều chạy trực tiếp qua venv trong nhiều terminal riêng, không có orchestration tự động.
- **B1.2 chưa có benchmark precision/recall chính thức:** pipeline Hybrid Search chứng minh hoạt động đúng về mặt logic qua 2 câu test thủ công (1 câu mơ hồ đúng kỳ vọng lọc hết, 1 câu khớp rõ cho kết quả tốt), nhưng CHƯA chạy bộ 20 câu hỏi test chính thức theo đề bài để đo top-5 precision (>=80%) và so sánh recall có/không Graph Expansion (>=15% cải thiện) — thiếu bằng chứng định lượng đầy đủ.
- **B1.1/B1.3 thiếu bằng chứng đầy đủ:** xác nhận Neo4j có 220 Chunk node thật (không rỗng), nhưng chưa breakdown theo Menutem/FAQ-chunk/Document-chunk để đối chiếu 3 ngưỡng đề bài (>=100/>=150/>=50); B1.3 (Ingestion pipeline, Entity Deduplication, Watch Mode) hoàn toàn chưa được kiểm chứng trong phiên làm việc này.

5. Kết luận và khuyến nghị

Tổng hợp tiêu chí nghiệm thu Phần B:

Tiêu chí	Kết quả	Ghi chú
B1.1 Graph DB schema (MenuItem/Chunk/Entity + relationships)	PASS 1 phần	Schema đúng
B1.1 Node count (>=100 Menu / >=150 FAQ / >=50 Document)	CHƯA ĐỦ BẰNG CHỨNG	220 Chunk t
B1.2 Vector Index + Hybrid Search pipeline	PASS	Demo thật:
B1.2 Top-5 precision >= 80% / Graph Expansion recall +15%	CHƯA ĐO	Cần bộ 20 c
B1.3 Ingestion pipeline, Entity Dedup, Watch Mode	CHƯA KIỂM CHỨNG	Chưa xem l
Zero Hallucination (B1 + B2.4)	FAIL nghiêm trọng	Có bằng ch
B2.1 Serving qua SGLang	PASS 1 phần	Generator P
B2.1 Docker Compose containerization	FAIL — CHƯA CÓ	Chỉ Neo4j ch
B2.2 SSE Streaming + [DONE] + Clause-level	PASS (code review)	Chưa test e
B2.3 Caching đa tầng (Model/Graph/KV/Session/Semantic)	PASS (thiết kế + 1 phần chạy thật)	GraphCache
B2.4 TTFT <= 200ms Đạt	FAIL	3209.7ms đ
B2.4 Total Pipeline Latency <= 1.5s Đạt	CHƯA ĐO SẠCH	Số đo đêm r

Kết quả demo video thật (GraphRAG retrieval nối tiếp SSE streaming):

Bước	Kết quả thật
Câu hỏi	"Cho em hỏi cà phê sữa đá size L giá bao nhiêu vậy?"
B1.2 Hybrid Search	3 chunk, rerank=0.994/0.936/0.914 (đều >=0.7) — đúng cả 3 đều liên quan cà phê
Context đưa vào Generator	3 dòng mô tả món thật kèm giá (48000đ) lấy từ Neo4j
TTFT	3209.7 ms
Câu trả lời	"Cà phê sữa đá size L giá 48000đ." — ĐÚNG khớp chính xác dữ liệu thật, không bịa
Tổng thời gian	4376.9 ms

Khi context đầy đủ và liên quan, Generator trả lời chính xác, không hallucination — đối lập rõ rệt với trường hợp context rỗng ở mục 2.4/Failure Analysis. Điều này càng khẳng định vấn đề nằm ở việc THIẾU cơ chế xử lý fallback khi retrieval không tìm được gì liên quan, không phải do Generator luôn luôn bịa.

Khuyến nghị cải thiện:

- Ưu tiên 0 (khẩn cấp) — thêm hard guardrail chống hallucination khi context rỗng:** trước khi gọi Generator, kiểm tra nếu context block rỗng (0 chunk qua ngưỡng rerank) thì trả thẳng câu trả lời cố định kiểu "Mình chưa có thông tin chính xác về việc này, để mình hỏi lại quây giúp anh/chị nhé" — KHÔNG gọi Generator sinh câu trả lời tự do trong trường hợp này. Đây là fix bắt buộc vì Zero Hallucination là tiêu chí tối thượng theo đề bài.

- **Ưu tiên 1 — đo lại TTFT/Pipeline Latency ở trạng thái warm:** chạy benchmark với nhiều request liên tiếp (loại bỏ request đầu tiên khỏi thống kê, hoặc warm-up trước 3-5 request), tách riêng thời gian load model (một lần) khỏi latency pipeline mỗi request, để có số liệu TTFT/Total Latency phản ánh đúng hiệu năng ổn định, so sánh công bằng với ngưỡng đề bài.
- **Ưu tiên 2 — đưa Router vào serve qua SGLang cùng Generator (đúng kiến trúc dual-model đề bài):** giải quyết đồng thời 2 vấn đề — đúng kiến trúc đề bài yêu cầu, và giảm tranh chấp VRAM không kiểm soát (nguyên nhân OOM đã ghi nhận ở Phần A).
- **Ưu tiên 3 — container hóa bằng Docker Compose:** viết docker-compose.yml khởi động theo đúng thứ tự Neo4j -> SGLang Generator+Router -> FastAPI api_server, đúng yêu cầu B2.1.
- **Ưu tiên 4 — chạy benchmark định lượng còn thiếu cho B1.2:** chuẩn bị bộ 20 câu hỏi test (có nhãn relevant/không relevant thủ công), đo top-5 precision và so sánh recall có/không Graph Expansion để có bằng chứng định lượng đầy đủ thay vì chỉ demo định tính.
- **Ưu tiên 5 — xác nhận B1.1/B1.3:** chạy Cypher đếm riêng Menultem/FAQ-chunk/Document-chunk theo field phân loại (source/category) để đối chiếu 3 ngưỡng đề bài; review lại code ingestion (entity dedup Jaccard, watch mode) để xác nhận PASS/FAIL chính thức cho B1.3.
- **Ưu tiên 6 — test end-to-end qua chính api_server.py:** chạy uvicorn api_server:app --port 8080 rồi gọi thật qua curl/test_sse_client.py vào /v1/chat/stream, thay vì chỉ test logic streaming trong script riêng — đảm bảo toàn bộ FastAPI layer (không chỉ phần logic gọi SGLang) hoạt động đúng.